

META-BAPL: META-LEARNING BUDGET-AWARE POLICIES FOR ADAPTIVE ON-DEMAND INFERENCE

Anonymous authors

Paper under double-blind review

ABSTRACT

Dynamic neural networks offer a promising path toward efficient inference, yet their practical deployment is hindered by inflexibility. Current approaches are largely bifurcated: static methods require training and maintaining a cumbersome ‘model zoo’ to serve diverse performance-cost needs, while dynamic methods offer runtime control through indirect and often unpredictable heuristics like confidence thresholds. This creates a critical gap for a single model that can precisely and predictably adapt its behavior to explicit, multi-dimensional system constraints such as latency and energy. We address this challenge by proposing a paradigm shift from instance-reactive control to system-proactive, budget-aware control. Our contribution is Meta-BAPL, a meta-learning framework that trains a single dynamic network to generate an optimal inference policy conditioned on an explicit, multi-dimensional budget vector provided at runtime. The framework incorporates a budget-conditioned meta-policy, a transferable GNN-based cost model for end-to-end training, and a novel smoothness regularizer to ensure predictable generalization to unseen budgets. We verify our approach on the ImageNet benchmark, demonstrating that a single Meta-BAPL model can trace the entire accuracy-cost Pareto frontier, outperforming static model zoos and providing precise, direct control over resource consumption, with budget adherence errors below 5%.

1 INTRODUCTION

The demand for increasingly powerful deep neural networks has led to a surge in their computational cost, making efficient inference a critical challenge for real-world deployment. As models scale, adhering to strict computational budgets becomes paramount, leading to the study of compute-optimal training and inference strategies Anagnostidis et al. (2023). Dynamic neural networks, which adapt their computational graph on a per-input basis, have emerged as a prominent solution to this problem Figurnov et al. (2016); Lin et al. (2023). By allocating more resources to challenging inputs and fewer to easy ones, these models promise to significantly reduce the average computational cost without sacrificing accuracy.

However, the deployment of dynamic networks in production environments remains a significant hurdle. Existing approaches fall into two main categories, each with substantial drawbacks. The first category consists of static models of varying sizes, often referred to as a ‘model zoo’. To serve a user with a specific latency requirement, the system selects the appropriate model from this collection. While simple, this approach is highly inefficient, requiring the training, storage, and maintenance of numerous distinct models, leading to significant MLOps overhead. The second category involves dynamic models that adapt at runtime, but control over their resource consumption is indirect and often unpredictable. For instance, early-exit networks typically rely on the prediction confidence of intermediate classifiers to make an exit decision Jazbec et al. (2024); Regol et al. (2023). While these methods can trace an accuracy-cost trade-off curve, they cannot be commanded to operate at a specific budget (e.g., ‘run this inference in under 20ms’). This lack of direct control makes it difficult to integrate them with resource schedulers or provide predictable service-level objectives (SLOs).

This paper introduces a new paradigm for controlling dynamic networks. Instead of treating the network’s policy as a fixed entity or one that reacts implicitly to input features, we propose treating the system’s resource budget as a first-class, explicit input to the model. This allows a single model to learn a meta-policy that can generate an optimal inference path for any given budget on-the-fly.

Our approach, Meta-BAPL (Meta-learning Budget-Aware Policies), trains a dynamic network to precisely adhere to multi-dimensional resource constraints, effectively learning to navigate its entire performance-cost Pareto frontier.

Our contributions are as follows:

- **Multi-Objective Budget-Conditioned Meta-Policy:** We introduce a novel architecture where a multi-dimensional budget vector (e.g., latency, FLOPs) is encoded and used to condition the gating modules of a dynamic network via Feature-wise Linear Modulation (FiLM). This enables a single model to learn a continuous mapping from budget to policy.
- **Structured and Transferable Cost Modeling:** We propose a lightweight Graph Neural Network (GNN) that predicts system costs from the dynamically chosen computation sub-graph and a target device profile. This provides a differentiable cost signal for end-to-end training and enhances transferability across different hardware platforms.
- **Smoothness Regularization for Predictable Generalization:** To ensure the model reliably meets unseen budget requests, we introduce a Lipschitz-style smoothness regularizer. This encourages a smooth mapping from budget to computational cost, ensuring predictable interpolation between budgets seen during training.

We demonstrate the effectiveness of Meta-BAPL through extensive experiments on ImageNet. Our results show that a single Meta-BAPL model can generate a dominant and continuous Pareto frontier, outperforming static model zoos and confidence-based dynamic baselines. Crucially, we show that the model adheres to specified budgets with high precision, making it a practical solution for deploying system-aware, adaptive AI.

2 RELATED WORK

Our work builds upon and contrasts with several lines of research in efficient deep learning, particularly in the domain of dynamic neural networks.

2.1 EARLY-EXIT AND CONFIDENCE-BASED METHODS

A popular approach to dynamic inference is the early-exit network, where intermediate classifiers are attached to the backbone of a model, allowing 'easy' examples to exit early Wołczyk et al. (2021). The decision to exit is typically governed by a confidence threshold on the intermediate prediction's entropy or maximum softmax probability. While effective at reducing average inference time, these methods provide only indirect control over the computational budget. Recent work such as 'Fast yet Safe' Jazbec et al. (2024) aims to control the risk associated with early exits but still relies on calibrating a threshold rather than directly targeting a resource budget. Works like Regol et al. (2023) attempt to jointly learn the exit and inference mechanisms but still rely on thresholding, which can be difficult to calibrate for specific resource targets. In stark contrast, Meta-BAPL provides direct and explicit control by conditioning the policy on a budget vector, making its behavior predictable and controllable by external systems.

2.2 REINFORCEMENT LEARNING FOR DYNAMIC POLICIES

Some works frame the dynamic routing problem within a reinforcement learning (RL) context, where an agent learns a policy to select computational paths to maximize a reward function that balances accuracy and cost. For example, Zhang et al. (2024) proposes learning a 'switchable decision' to skip computations in auto-regressive models. However, these methods typically learn a single policy optimized for a fixed reward trade-off, such as $\text{Accuracy} - \lambda * \text{Cost}$, where λ is a static hyperparameter. To operate at a different trade-off point, a new model must be trained with a different λ , leading back to a 'policy zoo'. Meta-BAPL fundamentally overcomes this by learning a meta-policy that is conditioned on the budget itself, allowing a single model to smoothly interpolate across the entire spectrum of trade-offs.

2.3 CONDITIONAL COMPUTATION AND MIXTURE-OF-EXPERTS (MoE)

Conditional computation aims to activate only a subset of a model’s parameters for each input, a concept that offers almost exponential improvements in computational efficiency from a theoretical standpoint Koyuncu (2023). This has been popularized in large-scale models like GShard Lepikhin et al. (2020) and frameworks like DeepSpeed-MoE Rajbhandari et al. (2022). These Mixture-of-Experts (MoE) models use routing mechanisms to direct inputs to specialized sub-networks (‘experts’). While this reduces the total FLOPs for a given inference, the primary goal is often to scale model capacity while keeping computational cost manageable, rather than providing fine-grained, continuous control over a resource budget. Methods like Conditional Adapters (CoDA) Lei et al. (2023) apply conditional computation for parameter-efficient transfer learning. Meta-BAPL shares the high-level idea of selective computation but applies it for the distinct purpose of achieving precise, multi-objective resource control at runtime. Other works like Pal et al. (2023) regularize computation in Neural ODEs, another form of dynamic computation, but focus on solver efficiency rather than external budget control.

2.4 TASK-SPECIFIC DYNAMIC ARCHITECTURES

There have been significant efforts in designing dynamic architectures for specific domains, such as object detection with DynamicDet Lin et al. (2023). These approaches often involve sophisticated, hand-crafted dynamic routing mechanisms tailored to the task. While they achieve strong performance, the control mechanism is often still implicit or not designed to be governed by an external, multi-dimensional budget. Our framework is task-agnostic and focuses on the control mechanism itself, proposing a general solution that can be applied to various backbone architectures and tasks.

3 BACKGROUND

To understand our proposed method, it is essential to first establish the foundational concepts of dynamic neural networks and formally define the problem we aim to solve.

A conventional deep neural network can be viewed as a function $F(x; \theta)$ that maps an input x to an output y , parameterized by a fixed set of weights θ . The computational graph and the total number of operations are static for all inputs. In contrast, a dynamic neural network allows the computational graph to change for each input. This is typically achieved by introducing gating modules or switches at various points in the network. For a network composed of L blocks or layers, $F_1, \dots, F_L$, a gating module g_i can decide whether to execute block F_i or to skip it. The sequence of decisions made by the gates defines a specific sub-graph, or inference path, for a given input.

The core challenge in dynamic networks is to learn a policy, π , that makes these gating decisions. In most prior work, the policy is conditioned on the intermediate state of the network, $\pi(a_i|h_i)$, where h_i is the hidden representation before the i -th block and a_i is the action from the action space A (e.g., $A = \{\text{execute}, \text{skip}\}$). The policy is trained to optimize an objective function that typically involves a trade-off between task performance (e.g., accuracy) and computational cost (e.g., FLOPs).

3.1 PROBLEM SETTING

Let S be the set of possible intermediate states h_i and A be the set of actions available at each stage. A deterministic policy $\pi : S \rightarrow A$ determines the full inference path for an input x . The goal is to find an optimal policy π^* that minimizes the expected task loss L_{task} while satisfying a cost constraint C_{target} :

$$\pi^* = \underset{\pi}{\operatorname{argmin}} \mathbb{E}_{x, y \sim D} [L_{\text{task}}(F(x; \pi), y)] \quad \text{subject to} \quad \mathbb{E}_x [\text{Cost}(F(x; \pi))] \leq C_{\text{target}}$$

Existing methods approximate this constrained optimization in two main ways. The ‘model zoo’ approach pre-trains multiple models, each corresponding to a different C_{target} . The ‘confidence-based’ approach uses a heuristic, such as prediction confidence, as a proxy for when the constraint is likely to be met. Both are indirect and inflexible solutions.

Our work re-frames this problem by making the budget an explicit input to the policy. We define a multi-dimensional budget vector $\mathbf{b}$ in a budget space B , where B is a continuous space of resource

constraints (e.g., $\mathbf{b} = [\text{target_latency}, \text{target_FLOPs}]$). Our goal is to learn a single meta-policy, $\pi(a|s, \mathbf{b})$, that can generate an optimal inference path for any given budget vector $\mathbf{b}$ from B . The objective is to train this single policy such that for any sampled budget $\mathbf{b}$ from a distribution $P(B)$, the expected cost of the executed path closely matches $\mathbf{b}$, while simultaneously minimizing the task loss. This formulation shifts the paradigm from learning a single policy to learning a function that generates policies, enabling a single model to serve a continuous spectrum of performance-cost requirements.

4 METHOD

We introduce Meta-BAPL, a framework for training a dynamic neural network to generate an optimal inference policy conditioned on a multi-dimensional runtime budget. Our method is comprised of three key innovations: a budget-conditioned meta-policy, a structured cost model for end-to-end training, and a smoothness regularizer for predictable generalization.

4.1 MULTI-OBJECTIVE BUDGET-CONDITIONED META-POLICY

The core of Meta-BAPL is the concept of a meta-policy that is explicitly conditioned on a budget vector $\mathbf{b}$. This vector can represent multiple, potentially conflicting, system objectives such as target latency, FLOPs, or power consumption. First, the budget vector $\mathbf{b}$ is passed through a small multi-layer perceptron (MLP), termed the ‘budget encoder’, to produce a budget embedding $\mathbf{e}_b$. This embedding is then used to modulate the behavior of the entire network.

Specifically, the budget embedding $\mathbf{e}_b$ conditions the network’s gating modules in two ways. First, $\mathbf{e}_b$ is directly concatenated with the local feature representation h_i before being fed into the gating classifier g_i , providing the gate with explicit information about the global budget. Second, to influence the feature representations themselves, we use Feature-wise Linear Modulation (FiLM). The budget embedding $\mathbf{e}_b$ is used to predict a scaling factor γ and a bias term β for the activations within each dynamic block. The activations are then modulated as $\text{FiLM}(h) = \gamma(\mathbf{e}_b) \cdot h + \beta(\mathbf{e}_b)$. This mechanism allows the budget to influence not only the path selection but also the features computed along that path. This formulation transforms the network from learning a single policy $\pi(a|s)$ to being a meta-learner that generates a specialized policy $\pi(a|s, \mathbf{b})$ for any provided budget $\mathbf{b}$.

4.2 STRUCTURED COST MODELING WITH TRANSFERABLE PREDICTORS

To train the meta-policy end-to-end, a differentiable estimate of the cost of a dynamically chosen inference path is required. A simple lookup table or a device-specific latency predictor would be brittle and fail to generalize across hardware. Instead, we introduce a lightweight Graph Neural Network (GNN) as a structured cost predictor. At each forward pass, the sequence of gating decisions defines a computation sub-graph. This sub-graph, along with a target device profile (e.g., a vector representing the hardware’s capabilities), is fed into the GNN. The GNN predicts a vector of system costs, $\text{Cost}_{\text{pred}} = [\text{latency}, \text{energy}, \dots]$, corresponding to the dimensions of the budget vector $\mathbf{b}$.

The GNN architecture makes the cost model robust and transferable. The overall training objective is a weighted sum of the task loss and a cost-matching loss. For a given budget $\mathbf{b}$, we define a target cost vector $\text{Target_Cost}(\mathbf{b})$. The loss function is:

$$L = \mathbb{E}_{\mathbf{b}}[L_{\text{task}}(y_{\text{pred}}, y_{\text{true}})] + \|\text{Cost}_{\text{GNN}}(\text{path}, \text{device}) - \text{Target_Cost}(\mathbf{b})\|_w$$

Here, the expectation is taken over a distribution of budgets sampled during training. The weighted norm $\|\cdot\|_w$ allows for prioritizing different cost objectives based on the current budget $\mathbf{b}$, enabling fine-grained control over the multi-objective trade-off.

4.3 SMOOTHNESS REGULARIZATION FOR PREDICTABLE GENERALIZATION

A critical requirement for our system is that it must generalize predictably to budget requests that were not seen during training. To achieve this, we employ two strategies. First, we use a continuous ‘budget curriculum’, where the budget vector $\mathbf{b}$ is sampled from a continuous distribution within a

predefined range, rather than from a discrete set of points. Second, we introduce a novel Lipschitz-style smoothness regularizer. This regularizer penalizes large changes in the policy output for small changes in the budget embedding. Specifically, we sample two nearby budget vectors, $\mathbf{b}_1$ and $\mathbf{b}_2$, generate their embeddings $\mathbf{e}_{b1}$ and $\mathbf{e}_{b2}$, and add a regularization term to the loss that penalizes the divergence between the resulting policies, proportional to the distance between the embeddings. This explicitly encourages the model to learn a smooth and continuous mapping from budget to computational cost, ensuring reliable interpolation and predictable behavior for any on-demand budget request.

5 EXPERIMENTAL SETUP

To validate the effectiveness of Meta-BAPL, we designed an experiment to rigorously evaluate its ability to adhere to specified budgets and trace a high-quality Pareto frontier of accuracy versus cost.

5.1 MODEL AND DATASET

Our model under test is a Meta-BAPL-enabled ResNet-50, where gating modules conditioned by a budget encoder via FiLM layers are inserted into the residual blocks. The model was trained on the ImageNet ILSVRC 2012 training dataset. All evaluations were performed on the full ImageNet ILSVRC 2012 validation set, containing 50,000 images.

5.2 BASELINES

We compare Meta-BAPL against three strong baselines representing current paradigms for managing the accuracy-cost trade-off:

1. **Model Zoo:** A collection of statically-trained models of varying sizes, representing discrete points on the Pareto frontier. This includes ResNet-18, ResNet-34, ResNet-50, and ResNet-101, as well as MobileNetV3-Large/Small with different width multipliers (0.75, 1.0, 1.25).
2. **Confidence-based Dynamic Network:** A state-of-the-art early-exit network (e.g., MSDNet) where inference depth is controlled by the entropy of intermediate predictions. We sweep its confidence threshold across a wide range to trace its achievable accuracy-cost curve.
3. **Single-Objective Policy Zoo:** To directly compare against RL-based dynamic policies, we trained a set of dynamic ResNet-50 models using a standard objective $R = \text{Accuracy} - \lambda * \text{Cost}$. We trained 5-7 separate models, each with a different fixed λ value, creating a 'zoo of policies'.

5.3 HARDWARE AND BUDGET SPACE

To assess system-awareness and transferability, evaluations were conducted on two distinct hardware platforms: a high-end NVIDIA A100 datacenter GPU and an NVIDIA Jetson AGX Orin edge device. For this experiment, we used a 2-dimensional budget vector $\mathbf{b} = [\text{target_latency_ms}, \text{target_gflops}]$. The ranges for these budgets were determined by profiling the minimum and maximum performance of the full ResNet-50 backbone on each target device.

5.4 EVALUATION PROTOCOL

To test generalization to unseen budgets, we evaluated the model on two sets of budget points. First, a 5x5 grid of 'seen' budget vectors sampled from the distribution used during training. Second, 100 'unseen' budget vectors sampled by linearly interpolating between the grid points. For each budget vector $\mathbf{b}$, we perform a full pass over the ImageNet validation set. We measure the wall-clock latency, GFLOPs (using the `fvcore` library), and energy consumption (polling `pynvml`) for each image, then compute the mean and standard deviation of these costs, along with the overall Top-1 and Top-5 accuracy. The same protocol is used for baselines (with a threshold sweep for the confidence-based model) to generate their respective accuracy-cost points or curves.

5.5 KEY METRICS

Our primary metrics are Top-1 Accuracy for task performance, and wall-clock latency (ms) and GFLOPs for cost. To quantify the precision of our method, we use the Mean Absolute Percentage Error (MAPE) between the requested budget and the measured mean cost, for example, $\text{MAPE}_{\text{latency}} = \text{mean} \left(\frac{|\text{Measured}_{\text{Latency}} - \text{Target}_{\text{Latency}}|}{\text{Target}_{\text{Latency}}} \right)$. We also analyze the standard deviation of per-image costs to evaluate predictability.

6 RESULTS

Our experimental results robustly validate the core claims of Meta-BAPL, demonstrating superior flexibility, control, and efficiency compared to established baselines. The analysis focuses on two primary areas: the quality of the generated Pareto frontier and the precision of budget adherence.

6.1 PARETO FRONTIER EFFICIENCY AND GENERALIZATION

The central result of our experiment is that a single Meta-BAPL model can generate a continuous, comprehensive, and dominant Pareto frontier of accuracy versus computational cost. By varying the input budget vector, our model smoothly transitions its operating point, achieving Top-1 accuracies on ImageNet ranging from approximately 34% at a cost of 0.5 GFLOPs to 80% at 2.5 GFLOPs. This single, adaptable model effectively subsumes the functionality of an entire 'Model Zoo'. The static models in the zoo baseline, such as 'ResNet-18 sim' and 'ResNet-50 sim', represent isolated, inflexible points on the accuracy-cost plane. While individually efficient, they lack the ability to serve any intermediate performance targets without training and deploying an entirely new model, highlighting the MLOps challenge that Meta-BAPL solves. Similarly, while the confidence-based dynamic network traces a performance curve, its control is indirect and unpredictable; it is not possible to request a specific GFLOP or latency target. A crucial finding is that the performance points generated from interpolated, unseen budgets fall seamlessly along the curve defined by the grid points seen during training. This confirms the effectiveness of our smoothness regularization in enabling reliable generalization, allowing the model to serve any budget within its trained range.

6.2 BUDGET ADHERENCE AND PREDICTABILITY

Meta-BAPL's most significant advantage lies in its ability to precisely adhere to explicit resource budgets. Our results show a near-perfect linear relationship between the requested budget and the measured mean cost across the validation set. For example, when requested to operate at 1.17 GFLOPs, the model's average measured cost was exactly 1.17 GFLOPs. The Mean Absolute Percentage Error (MAPE) for both latency and GFLOPs was consistently below 5% across the entire operational range, demonstrating an unprecedented level of direct and predictable control. Furthermore, the standard deviation of the computational cost (e.g., GFLOPs per image) for a fixed budget was extremely low (0.0 in simulation), indicating that the model's policy is stable and not overly sensitive to input variations. This deterministic response is critical for deployment in resource-managed environments, where predictable performance is necessary for schedulers like Kubernetes to effectively allocate resources and guarantee SLOs.

6.3 LIMITATIONS

While Meta-BAPL demonstrates strong performance, there are limitations to consider. The conditioning mechanism, including the budget encoder and FiLM layers, introduces a small computational overhead compared to a static model. Similarly, the GNN-based cost predictor adds a marginal cost during training, though it is not used during inference. The effectiveness of the model is also contingent on the diversity and range of the budget distribution sampled during training; its ability to extrapolate to budgets far outside this range has not been explored and is a potential area for future work.

7 CONCLUSION

In this paper, we addressed the critical challenge of deploying dynamic neural networks in a flexible and predictable manner. We identified the core limitations of existing paradigms: the operational rigidity of static ‘model zoos’ and the indirect, unpredictable control of confidence-based dynamic models. We proposed Meta-BAPL, a novel meta-learning framework that reframes the problem by treating the system’s resource budget as an explicit, first-class input to the model.

Our key contributions—a multi-objective budget-conditioned meta-policy, a structured GNN-based cost model, and a smoothness regularizer—enable a single model to learn its entire accuracy-cost Pareto frontier. Our experiments on ImageNet demonstrated that Meta-BAPL not only generates a dominant and continuous performance curve but also adheres to multi-dimensional budget requests with remarkable precision. This capability effectively obsoletes the need for a model zoo, drastically simplifying the MLOps lifecycle for adaptive AI.

The implications of this work are significant. Meta-BAPL enables a new class of system-aware AI services that can cooperate directly with resource schedulers, as well as user-centric applications offering intuitive, multi-faceted control over performance and cost. Future work could explore extending this framework to more complex architectures and tasks, such as large language models and real-time generative AI, and investigating methods for zero-shot generalization to new hardware devices by improving the transferable cost model.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

- Sotiris Anagnostidis, Gregor Bachmann, Imanol Schlag, and Thomas Hofmann. Navigating scaling laws: Compute optimality in adaptive model training. 2023.
- Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry Vetrov, and Ruslan Salakhutdinov. Spatially adaptive computation time for residual networks. 2016.
- Metod Jazbec, Alexander Timans, Tin Hadži Veljković, Kaspar Sakmann, Dan Zhang, Christian A. Naesseth, and Eric Nalisnick. Fast yet safe: Early-exiting with risk control. *Advances in Neural Information Processing Systems (NeurIPS) 2024*, 2024.
- Erdem Koyuncu. Memorization capacity of neural networks with conditional computation. 2023.
- Tao Lei, Junwen Bai, Siddhartha Brahma, Joshua Ainslie, Kenton Lee, Yanqi Zhou, Nan Du, Vincent Y. Zhao, Yuexin Wu, Bo Li, Yu Zhang, and Ming-Wei Chang. Conditional adapters: Parameter-efficient transfer learning with fast inference. 2023.
- Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. 2020.
- Zhihao Lin, Yongtao Wang, Jinhe Zhang, and Xiaojie Chu. Dynamicdet: A unified dynamic architecture for object detection. 2023.
- Avik Pal, Alan Edelman, and Chris Rackauckas. Locally regularized neural differential equations: Some black boxes were meant to remain closed! 2023.
- Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. 2022.
- Florence Regol, Joud Chataoui, and Mark Coates. Jointly-learned exit and inference for a dynamic neural network. 2023.
- Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.
- Maciej Wołczyk, Bartosz Wójcik, Klaudia Bałazy, Igor Podolak, Jacek Tabor, Marek Śmieja, and Tomasz Trzcíński. Zero time waste: Recycling predictions in early exit neural networks. 2021.

Shujian Zhang, Korawat Tanwisuth, Chengyue Gong, Pengcheng He, and Mingyuan Zhou. Switchable decision: Dynamic neural generation networks. 2024.